SIRAV — Informe ejecutivo

Sistema Integral de Reservas y Alquiler de Vehículos

Integrantes: David Felipe Bejarano Montes y Jose David Rodriguez
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte · Bases de Datos 2 — Proyecto Final
Fecha: Bogotá D.C., noviembre de 2026 · Repositorio: github.com/DavidBejarano309/SIRAV

1. Problema

Una empresa pequeña de alquiler de vehículos administra reservas, entregas, devoluciones y cobros con hojas de cálculo y mensajes. El riesgo central es asignar el mismo vehículo a dos clientes en fechas que se cruzan, sobre todo cuando dos personas reservan al mismo tiempo. Además, la información de las inspecciones (daños, fotos y observaciones) no tiene una estructura fija y se pierde o se registra de forma inconsistente, y no hay trazabilidad de quién cambió el estado de una reserva ni de cuánto se cobró.

El proyecto se planteó como un caso de estudio de integridad y concurrencia: la regla de negocio más importante debía cumplirse en la base de datos, no solo en la interfaz, y la solución debía combinar un motor relacional y uno documental según la naturaleza de cada dato (Sadalage y Fowler, 2012; Stonebraker, 2010).

2. Solución

SIRAV es una aplicación web que cubre el ciclo completo: catálogo con disponibilidad por fechas, reserva, confirmación, entrega con inspección, devolución con inspección y cálculo de cargos, pago simulado, historial y reportes, con tres perfiles (administrador, operador y cliente).

- **PostgreSQL es la fuente de verdad.** El esquema tiene 13 tablas en tercera forma normal. Las operaciones del negocio las ejecutan cinco procedimientos (`sp_`), apoyados en tres funciones (`fn_`) y cuatro triggers (`trg_`). Cada error de negocio tiene su propio código SQLSTATE (SR001 a SR007) y nombre (`e_reserva_solapada`, `e_transicion_invalida`…). El usuario de la aplicación no puede escribir directamente en reservas, alquileres, pagos ni auditoría.
- **MongoDB guarda lo que es documento.** Las inspecciones de entrega y devolución, con daños y evidencias embebidos y versionadas, y una bitácora de eventos. Una consulta compleja (`$elemMatch`) y tres agregaciones alimentan los indicadores de daños y uso.
- **Aplicación.** API REST en Spring Boot 3.5 (Java 21) con JWT y BCrypt, e interfaz en React 19. Seis reportes SQL (R1 a R6) usan ventanas, `generate_series`, CTE y anti-joins.

3. Arquitectura

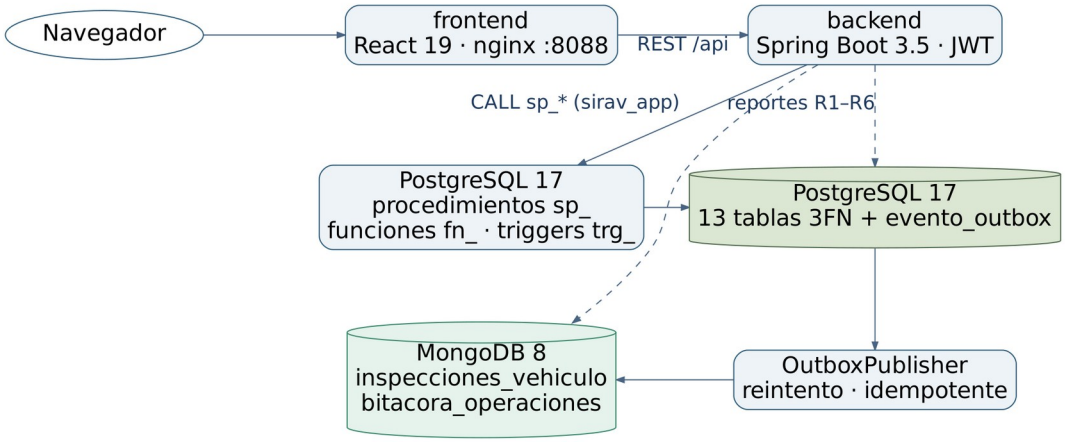


Figura 1. Despliegue de SIRAV: cuatro contenedores Docker en una VM Ubuntu Server 24.04. Líneas discontinuas: lecturas directas (reportes SQL; CRUD, consulta y agregaciones de MongoDB).

La interfaz llama a la API, y la API invoca los procedimientos con el usuario de mínimo privilegio `sirav_app`. Cuando una operación produce un documento (una inspección o un evento), el procedimiento lo escribe en la tabla `evento_outbox` dentro de la misma transacción. Un publicador lo copia luego a MongoDB, con reintento y sin duplicados. Así, si MongoDB no responde, no se pierde la inspección ni se deja un alquiler a medias. Este es el problema de consistencia entre almacenes que Laigner et al. (2021) identifican como central en sistemas con varias bases. Solo el puerto web (8088) queda publicado; las bases de datos no son accesibles desde fuera de la VM.

4. Decisiones técnicas

Tabla 1. Principales decisiones

Decisión	Alternativa descartada	Razón
Restricción de exclusión GiST sobre daterange [inicio, fin)	Validación solo en la aplicación o con triggers	Es la única forma declarativa de expresar "sin reservas vigentes solapadas". Se cumple aunque dos transacciones lleguen a la vez; se complementa con SELECT … FOR UPDATE sobre el vehículo.
PostgreSQL	MySQL 8.4 y MariaDB 11.8	Puntaje 5,00 frente a 2,80 y 3,40 en la matriz de selección: tipos de rango, exclusión condicional, PL/pgSQL con SQLSTATE propios y DDL transaccional.
MongoDB solo para inspecciones y bitácora	Todo en un solo motor	Las inspecciones tienen listas variables que se leen completas (patrones Embedding y Polymorphic). El dinero y los estados exigen transacciones ACID (Davoudian et al., 2018; Cattell, 2010).
Patrón outbox	Escribir en ambos motores desde la API	Sin transacción distribuida, la doble escritura puede dejar datos inconsistentes. El outbox da atomicidad local y publicación idempotente.
Identidad del cliente en un solo lugar	Copiar nombre y correo en cliente	La copia violaba 3FN y causaba anomalías de actualización (Codd, 1970). Una restricción CHECK y la vista v_cliente_contacto lo resuelven.
Docker Compose en Ubuntu Server 24.04	Servicios systemd instalados a mano; Ubuntu 26.04	Mismas versiones en la VM, en los portátiles y en la integración continua. MongoDB 8 no soporta Ubuntu 26.04.
React + API REST	Thymeleaf (propuesta v1.1)	Separa la interfaz de la lógica y permite probar la API sola. Se registró como cambio en la propuesta v1.2.

5. Resultados

Tabla 2. Resultados al cierre

Componente	Resultado verificable
Requerimientos	Los 8 funcionales y RNF-01, RNF-02 y RNF-04 cumplidos con trazabilidad a tablas, endpoints y pruebas. RNF-03: despliegue automatizado y documentado; falta instalarlo en la VM.
Concurrencia (RNF-01)	Prueba con dos hilos que reservan el mismo vehículo a la vez: exactamente un ganador y un rechazo HTTP 409.
Pruebas automáticas	11 del backend (8 de integración contra PostgreSQL y MongoDB reales), 3 de interfaz y 3 de navegador. Todas aprobadas en GitHub Actions.

Componente	Resultado verificable
Despliegue	Workflow que construye las cuatro imágenes en Ubuntu 24.04 y recorre el flujo completo por la API. También respalda (pg_dump y mongodump), restaura en otra base, compara conteos y reinicia sin perder datos.
Seguridad	BCrypt, JWT de 30 minutos revocable, tres perfiles en la aplicación y tres roles en la base (sirav_app, sirav_reportes, sirav_admin), y procedimientos SECURITY DEFINER.
Documentación	Propuesta v1.2, supuestos, requerimientos, riesgos, Gantt y RACI. Diagramas Chen, EER y Crow's Foot generados del catálogo real. Diccionario con verificación de 3FN, reportes R1 a R6, modelo documental y guía de instalación.

Pendiente: instalar la VM en el computador del equipo y repetir allí la verificación (SIRAV-15), y grabar el video demostrativo (SIRAV-37).

6. Lecciones aprendidas

- **Las reglas críticas van en la base.** La restricción de exclusión impidió solapamientos que una validación en la aplicación no detecta bajo concurrencia. Las excepciones con nombre y código hicieron los errores comprensibles para el usuario y para las pruebas.
- **Probar contra motores reales encuentra errores reales.** La integración continua detectó dos fallas que no aparecían en los equipos. El usuario de MongoDB tenía permisos sobre la base equivocada. Y el servidor de CI, en UTC, hacía fallar las pruebas cada noche entre las 19:00 y las 24:00 de Bogotá, porque la base calcula las fechas en hora local.
- **Una sola fuente de verdad también aplica al proyecto.** Durante el desarrollo coexistieron dos versiones del esquema (scripts sueltos y aplicación). Unificar costó trabajo; desde entonces cada diagrama y el diccionario se generan del catálogo real.
- **La normalización se nota en los datos.** Guardar el nombre del cliente en dos tablas parecía práctico, pero producía datos contradictorios al actualizar. La consistencia volvió al aplicar 3FN con una restricción que la hace cumplir.

7. Conclusiones

SIRAV demuestra que un prototipo académico puede tener integridad de nivel profesional si las reglas se expresan donde no se pueden evitar: restricciones declarativas, procedimientos transaccionales y privilegios mínimos en PostgreSQL. MongoDB aporta flexibilidad donde la estructura varía, sin sustituir la fuente de verdad; el patrón outbox mantiene ambos motores coherentes sin transacciones distribuidas. El aislamiento por instantáneas de PostgreSQL (Ports y Grittner, 2012), junto con el bloqueo explícito del vehículo, sostiene la regla principal ante accesos simultáneos.

El despliegue con Docker Compose y su verificación automática hacen la instalación reproducible. Como trabajo futuro quedan la instalación en la VM del equipo, una prueba de carga con volumen real y precalcular las agregaciones documentales (patrón Computed) si los datos crecen.

Referencias

Cattell, R. (2010). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, *39*(4), 12–27. https://doi.org/10.1145/1978915.1978919

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685

Davoudian, A., Chen, L., & Liu, M. (2018). A survey on NoSQL stores. *ACM Computing Surveys*, *51*(2), 1–43. https://doi.org/10.1145/3158661

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Laigner, R., Zhou, Y., Salles, M. A. V., Liu, Y., & Kalinowski, M. (2021). Data management in microservices: State of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, *14*(13), 3348–3361. https://doi.org/10.14778/3484224.3484232

Ports, D. R. K., & Grittner, K. (2012). Serializable snapshot isolation in PostgreSQL. *Proceedings of the VLDB Endowment*, *5*(12), 1850–1861. https://arxiv.org/abs/1208.4179

Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

Stonebraker, M. (2010). SQL databases v. NoSQL databases. *Communications of the ACM*, *53*(4), 10–11. https://doi.org/10.1145/1721654.1721659